Data Engineering
Pipeline delivery

Project handover

Pipeline Handover Document

Online store data pipeline: architecture, data quality rules, operating procedures and deliverables.

SAMPLE



Client	Example Online Store (fictional)
Project	Online store data pipeline, DAG `store_dwh_daily`
Prepared by	Data Engineer
Version	1.0
Date	17 October 2026
Status	Sample, based on the public demo project. All data is synthetic.

About this document

This document records how the `store_dwh_daily` pipeline is built and how to operate it. It is written for the client's team, including people who were not involved in the project, so that the pipeline can be run, fixed and extended without the original developer.

It describes the open-source demo project that accompanies this sample. Figures in section 3 come from a real run of that project on synthetic data.

Contents

1	Pipeline overview	3
2	Data quality	4
3	Run reporting	4
4	Operations runbook	5
5	Handover and sign-off	6

Prepared by

Role	Data Engineer at a large retail business; previously Business Analyst
Core tools	PostgreSQL, Microsoft SQL Server, Apache Airflow 3, Python, Git (Bitbucket, GitHub)
Time zone	GMT+7

My background in business analysis shapes how I work: requirements and the business questions behind the data are agreed in writing before development starts, and every rule in this document can be traced back to one of them.

Conventions

Term	Meaning in this document
Business date	The day of orders being processed. By default, the day before the run.
Run	One execution of the DAG for one business date
Blocking check	A quality rule that stops the run if it fails
Warning	A quality rule that is reported but does not stop the run

SAMPLE

1. Pipeline overview

1.1 Summary

Purpose	Load daily orders, customers and products into a star schema for reporting
Schedule	Daily at 02:00 UTC; can be triggered for any business date
Sources	Six CSV exports: categories, payment methods, customers, products, orders, order items
Target	PostgreSQL 16, schemas `stg`, `core`, `mart` and `audit`
Stack	Apache Airflow 3.3 (Task SDK), Python 3.12, SQLAlchemy

1.2 Tasks

Task or group	Description
`prepare_warehouse`	Creates schemas and tables if they do not exist
`generate_sources`	Demo only: writes synthetic source files. In production this is replaced by the client's file delivery.
`wave_1`	Categories, payment methods and customers
`wave_2`	Products, which reference categories
`wave_3`	Orders, which reference customers and payment methods
`wave_4`	Order items, which reference orders and products
`quality_checks`	Applies the rules in section 2.2; no retries, so failures surface immediately
`build_marts`	Rebuilds `mart.daily_sales` and `mart.category_sales` for the business date
`export_site_data`	Writes the JSON used by the reporting dashboard

Inside each wave, every table is a task group of two tasks: `extract_to_stg` (read, clean, stage) and `load_to_core` (upsert). Tasks retry twice, one minute apart, with a 20-minute timeout.

1.3 Design decisions

Decision	Reason
Upserts on natural keys	Any day can be rerun without creating duplicates
Load in dependency waves	Facts never load before the dimensions they reference
Rejected rows kept with a reason	Nothing is dropped silently; bad rows can be fixed and reloaded
Pipeline logic separate from the DAG	Rules are unit tested and can run without Airflow

SAMPLE

2. Data quality

2.1 Row-level rules, applied during staging

Rule	Action
Missing business key (customer, product, order)	Row rejected
Negative price or quantity ≤ 0	Row rejected
Order timestamp after the business date, or unknown status	Row rejected
Invalid email address	Email set to empty, customer kept
Duplicate rows in an export	Merged; the last occurrence wins

2.2 Checks after loading

Check	Condition	Severity
`no_negative_amounts`	No loaded item has quantity ≤ 0 or a negative price	Blocking
`no_future_orders`	No order is dated after the business date	Blocking
`orders_have_items`	Every order has at least one loaded item	Warning
`customers_have_email`	Customers without a usable email	Warning
`daily_volume_in_range`	Orders within ±50% of the previous 7-day average	Warning

3. Run reporting

3.1 Run for business date 16 October 2026

Table	Extracted	Rejected	Loaded
`dim_category`	8	0	8
`dim_payment_method`	4	0	4
`dim_customer`	1,509	1	1,500
`dim_product`	202	2	200
`fact_orders`	443	7	431
`fact_order_items`	724	17	707

Extracted minus rejected can exceed loaded because duplicate rows are merged (rule 2.1). Rejection reasons are listed in `audit.rejected_rows`; on this day the largest group was 11 order items whose order had been rejected.

SAMPLE

4. Operations runbook

Commands apply to Airflow 3.3. With Docker Compose, prefix them with `docker compose exec airflow`. A run processes the day before its logical date, so the run of 17 October loads orders from 16 October.

4.1 A run has failed

Open the failed task's log in the Airflow UI and resolve the cause. Then rerun only the failed tasks of that run:

```
airflow tasks clear store_dwh_daily -s 2026-10-17 -e 2026-10-17 --only-failed -y
```

4.2 Reprocess one business date

```
airflow dags trigger store_dwh_daily --conf '{"business_date": "2026-10-05"}'
```

4.3 Reprocess a period

Loads are upserts, so reprocessing does not create duplicates. Runs share the staging tables, so they run one at a time. This example reloads business dates 1 to 7 October:

```
airflow backfill create --dag-id store_dwh_daily --from-date 2026-10-02 --to-date 2026-10-08 --max-active-runs 1
```

4.4 A source changes its format

Rows that no longer match the expected columns fail the staging rules and appear in `audit.rejected_rows`; the volume check warns if most orders are missing. Update the mapping in `include/pipeline/transforms.py`, add a unit test for the new format, and reprocess the affected dates (4.2 or 4.3).

4.5 Escalation

Situation	Action
Failure alert received	Follow 4.1; if unresolved, contact the developer through the order page
Run succeeded but numbers look wrong	Check `audit.quality_results` and `audit.rejected_rows` for that date

SAMPLE

5. Handover and sign-off

5.1 Deliverables

Deliverable	Location
DAG and pipeline code	`dags/`, `include/pipeline/`
README with setup steps	Repository root
Unit tests and CI workflow	`tests/`, `.github/workflows/ci.yml`
Local environment (Airflow 3 and PostgreSQL)	`docker-compose.yaml`
Reporting dashboard	`docs/`, published with GitHub Pages
This handover document	`docs/handover.pdf`
Recorded video walkthrough	Shared with the delivery message

5.2 Support period

Defects in the delivered scope are fixed free of charge for 7 days after delivery. Changes to scope, new sources and new rules are quoted separately.

5.3 Revision history

Version	Date	Change
1.0	17 October 2026	Initial handover

5.4 Sign-off

	Name	Role	Date	Signature
Delivered by		Data Engineer		
Accepted by				

This is a sample document for a demo project. All data is synthetic and does not describe any real client or employer.

SAMPLE